VAEBERT: Generating Shapes from Captions using Variational Auto Encoders and BERT

Alexandru Munteanu Yi Wei George Z. Wei
Manning College of Information and Computer Sciences
University of Massachusetts Amherst
{amunteanu, yiw, gzwei}@umass.edu

Abstract

For many people, 3D shape creation is not very accessible because of the expertise one needs in 3D modeling software to generate such shapes. With the recent rise of the Metaverse, being able to quickly generate 3D objects to place into the virtual/augmented world is an important problem to address. Since natural language is a rich modality for describing the features of objects, we decided to endeavor in the task of text to shape generation. Specifically, we employed a Variational Auto Encoder (VAE) as well as a fine-tuned BERT or Bi-GRU with BERT embeddings as the text encoders. We perform qualitative and quantitative analysis for the performance of our approach, which we call VAEBERT. We have released our code at https://github.com/gzhihongwei/vaebert *to spur additional research on the text-to-shape generation task.*

1. Introduction

3D modeling is an integral part of any work related to 3D design, however the design process is complicated and hard to master. Having faster ways to model would increase accessibility for many portions of the design process, allowing even beginners to be able to delve into 3D design without needing to learn 3D modeling. For example, CAD softwares require knowledge such as parametric object building with primitives as well as learning how to use such a software, which usually has a steep learning curve. Even for experienced designers, having faster 3D model prototyping will save them time. Especially in light of the development of the Metaverse, it is reasonable to expect fast 3D modeling to become a hotly researched topic in the near future.

One way to increase accessibility would be to generate 3D models purely from natural language descriptions. By combining the power of natural language processing and existing works in 3D modeling, one can imagine an interface that can create baseline models from simple descriptions of human needs. In the realm of image generation, this area of text conditioned generation has recently taken large strides forward, such as DALL-E [13] and its successor DALL-E 2 [12]. Given this progress, we believed that the next step was text conditioned 3D shape generation, so we tackled this task here.

In this paper, we use the popular langauge model, BERT [3], and Variational Auto Encoders (VAEs) to attempt to solve this problem. Our model, called VAEBERT, uses a VAE to learn an encoding of 3D voxel models into a latent space. Then we fine-tuned BERT to learn encodings of the captions to the same latent space as the VAE, thus allowing us to piece together the language model with the decoder to create a text to shape pipeline.

2. Related Work

There has been previous work done on large-scale 3D shape datasets such as ShapeNet [1], which has 51,300 3D models with textures consisting of 55 classes. PartNet [9] is a 24-category, 26,671 model subset of ShapeNet with hierarchically segmented parts for each model (573,585 parts total).

For text-to-shape generation, Text2Shape [2] was the first work on this task. Specifically, Text2Shape learns shared embeddings between models of furniture and their text descriptions. They then used used the embeddings to generate colored shapes through a Wasserstein GAN. Following this work, StructureNet [8] represented shapes through a hierarchical n-array graph. Then it uses a VAE to encode the graphs as latent vectors and decode back to graphs which can reconstruct shapes. In "AI for 3D Generative Design" [4], voxels are created from the shape models and annotations from the part hierarchies of the top 11 classes in PartNet. Using a VAE and LSTM trained with SpaCy embeddings (based on GloVe embeddings) of the descriptions, the author created a system to generate voxels of shapes. Modern works in the text-to-shape generation space rely primarily on CLIP [11], such as [15, 5].

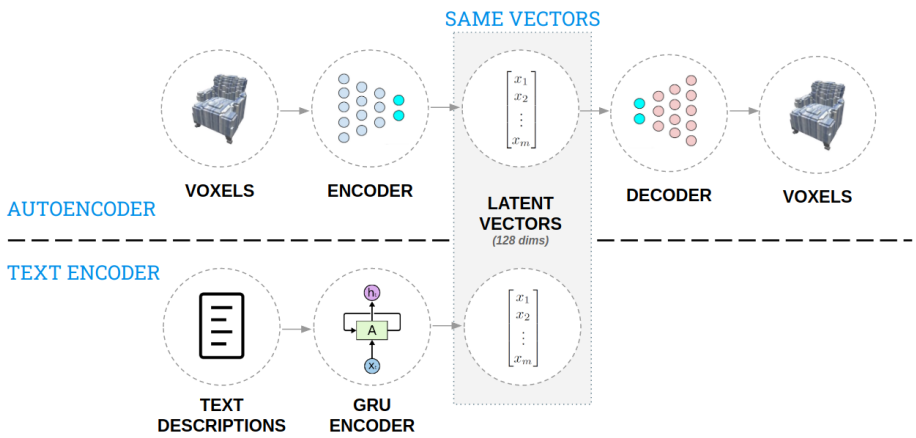


Figure 1: Training pipeline as given in [4]. We also experiment with a BERT encoder in additional to the Bi-GRU encoder.

Most recently, more progress has been made on text-to-image generation. DALL-E [13] was a monumental paper that showed promise in creating images that are faithful to the given text. GLIDE [10] has been shown to outperform DALL-E while using less than a third of the parameters by using diffusion models. However, DALL-E 2 [12] took another huge step over its predecessor by producing even more realistic images that reflect even difficult captions. Though we are not tackling text-to-image generation, we hope to create a system that can approach the task of text-to-shape generation.

3. Method

Since our project is essentially an extension of [4], the dataset preprocessing (§ 3.1) and overall architecture (§ 3.2) is very similar. However, we note down several differences especially with respect to the preprocessing and the language side of the pipeline.

3.1. Dataset Preprocessing

The original ShapeNet [1] dataset provides $128 \times 128 \times 128$ solid voxelizations of all shapes. The captions we use are the captions provided by [4] that were generated from the hierarchical annotations included in PartNet [9]. We only used shapes from the Table, Chair, Lamp, Faucet, Clock, Bottle, Vase, Laptop, Bed, Mug, and Bowl categories. We first checked calculated the sparsity of each shape's voxelization. Shapes with a sparsity less than 0.01 are removed from the dataset. If the sparsity of a remaining shape is less than 0.4, a $13 \times 13 \times 13$ Gaussian kernel is applied to it in order to increase its sparsity. The remaining shapes are centered along each axis based on the amount of empty space at the ends of the axes. Finally, 3D mean pooling is applied to all shapes with a kernel size of $2 \times 2 \times 2$ and a stride of 2 along all dimensions such that the resulting shapes become $64 \times 64 \times 64$ voxelizations.

3.2. Architecture

Figure 1 gives the overall pipeline of VAEBERT. Specifically, training assumes two stages. First, a VAE is trained on the preprocessed $64 \times 64 \times 64$ voxels down to a bottleneck/latent vector. After the VAE is trained, the text encoders will learn to regress to the bottleneck/latent vector learned by the encoder side of the VAE. For each of the textual encoders, we initialize the language representations with BERT embeddings. At inference time, only a textual description will be given, so the text encoder will predict a latent vector that the decoder side of the VAE will generate a voxelization of. We will talk about each component in turn.

3.2.1 Variational Auto Encoder

Given a $64 \times 64 \times 64$ voxelization, the VAE uses 5 3D convolutional layers and a linear layer to predict mean and log variance vectors, which can be used to form a latent representation using the normal distribution. Similarly to the encoder side, the decoder side uses a linear layer followed by 5 3D transpose convolutional layers to reconstruct the input. The standard VAE loss of KL divergence and reconstruction loss was used to train the model.

Since the codebase of [4] uses Keras/TensorFlow, the tricky part of reimplementing the VAE was the `padding="same"` used, which is not available for strides greater than 1 and entirely unavailable for the 3D transpose convolutional layers. Additionally, binary cross entropy with logits performs a mean across all dimensions for PyTorch but not TensorFlow, so that was another problem that was figured out while debugging.

3.2.2 BERT Text Encoder

Given a caption of a shape, the BERT tokenizer is used to create the inputs for BERT. After being processed through BERT, the representation of the `[CLS]` token was input into a simple linear layer to predict the latent vector. The representation of the `[CLS]` token was used because it was used for next sentence prediction during the pre-training of BERT, so the representation of this token is thought to encode the context of the whole caption [3].

3.2.3 Bi-GRU Text Encoder

Given a caption of a shape, the BERT tokenizer is used to create the inputs for a BERT where the weights are frozen. After being processed through BERT, all of the representations are fed to the Bi-GRU text encoder, which has 3 layers of bidirectional GRU layers with increasing hidden dimensionality. Finally, 4 linear layers bring down this concatenation of the forward and backward representations to the latent dimension size. As opposed to [4], we opted to use all representations produced by BERT instead of SpaCy embeddings since they are contextual embeddings and do not follow a bag-of-words approach. The representation of the `[CLS]` token was also used in the Bi-GRU because we believed that its use would be beneficial in contextualizing the early timesteps of the forward GRU and reinforcing the later timesteps in the backward GRU.

Like we found when reimplementing the VAE, Keras's `SpatialDropout1D` and s `stateful=True` for the GRU layers were not available in PyTorch. For the former issue, a hack[1] was used. For the latter issue, we return the hidden states of the Bi-GRU layers at each batch and fed those in as parameters into the model so that the hidden states get transferred between batches.

3.3. Implementation Details

For the VAE, we used a latent dimensionality of 128 and trained using a learning rate of 6e-4, weight decay of 1e-3, and batch size of 40 over 20 epochs using the Adam [6] optimizer. For the BERT based code, we used HuggingFace Transformers [16] for the Tokenizer and model itself. We chose to use the BERT base model as a foundation. For the BERT encoder, we fine-tuned with a learning rate of 5e-5 with a linear warmup ratio of 0.2 and linear decrease afterwards, weight decay of 1e-3, and batch size of 64 over 20 epochs using the AdamW [7] optimizer. For the Bi-GRU encoder, we trained with a learning rate of 1e-3, weight decay of 1e-3, and batch size of 64 over 1,000 epochs using the Adam [6] optimizer.

[1] https://discuss.pytorch.org/t/spatial-dropout-in-pytorch/21400/2

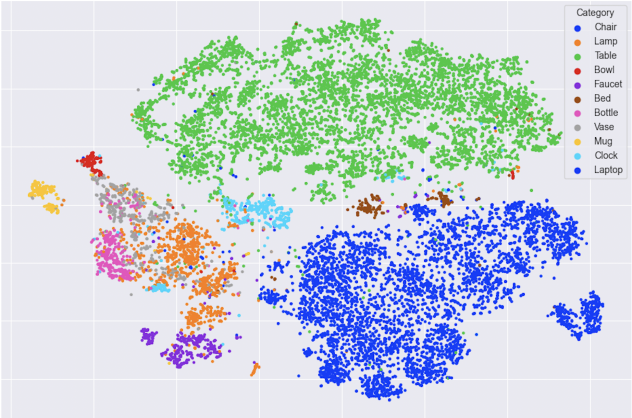


Figure 2: t-SNE plot of the latent vectors learned by the VAE.

4. Experiments

To sanity check that the VAE is learning properly, we plotted the t-SNE plot for the latent vectors as shown in Figure 2. We can see some clear clustering of the different shapes, so the VAE did learn some common representations each category.

At inference time, we used the Chamfer Distance to quantify how good the shape generation is given the caption as input. Chamfer distance $d(S_1, S_2)$ where S_1, S_2 are sets of binary voxels is given by

$$d(S_1, S_2) = \frac{1}{|S_1|} \sum_{x \in S_1} \min_{y \in S_2} \|x - y\|_2^2 + \frac{1}{|S_2|} \sum_{y \in S_2} \min_{x \in S_1} \|x - y\|_2^2, \tag{1}$$

which is the sum of the normalized L_2 norms of the closest from each point in S_1 to S_2 and vice versa.

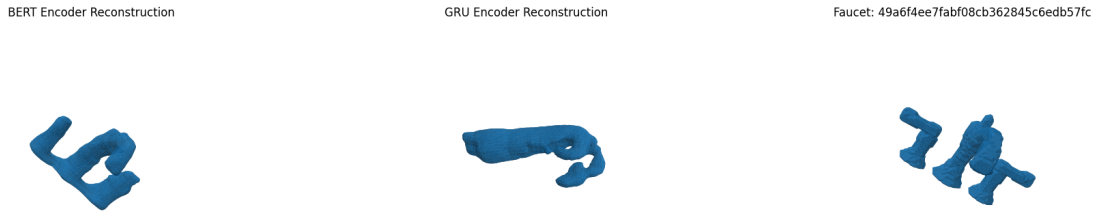
As we can see in Table 1, the VAE + BERT combination vastly outperformed the VAE + Bi-GRU combintation with BERT embeddings. This should come as no surprise as the BERT has much better semantic understanding of natural language from the start, so fine-tuning on the text to 3D shape generation task is no issue. On the other hand, the Bi-GRU struggles due to the vanishing and exploding gradient problems as well as the non-parallelizability of RNNs.

Even though the VAE + BERT combination is better on paper, how do the generations stack up qualitatively?

Table 1: Quantitative results of the two different approaches studied in this paper with respect to the Chamfer Distance. For this metric, lower is better.

Model	Chamfer Distance
VAE + BERT	**4.364**
VAE + Bi-GRU	5.244

(a) "A regular width straight chair and is low."



(b) "A faucet. it has two vertical supports."

(c) "A skinny laptop that has a screen and a touchpad and a keyboard."

Figure 3: Example generations from the test set. Left most column is VAE + BERT, middle column is VAE + Bi-GRU, and the right most column is the ground truth shape.

Figure 3 shows three such examples of captions, the generations from VAE + BERT, the generations from VAE + Bi-GRU, and the ground truth shape. Sometimes, both pipelines generated pretty well (Fig. 3a). Other times, either VAE + BERT or VAE + Bi-GRU produces more semantically correct/better generations (Fig. 3b and Fig. 3c).

5. Limitations

Due to time constraints, we couldn't run the Bi-GRU text encoder for the amount of epochs that was given by [4] (20,000 epochs versus the 1,000 epochs we trained for) and we did not have enough time to work on the adaptation of text-conditioned diffusion models to the 3D setting. Additionally, we couldn't find a proper implementation of Earth Mover's Distance (Wasserstein metric) that would compile and run successfully with the hardware available to us, and PyTorch3D [14], a sublibrary that makes PyTorch suitable for 3D vision/graphics, said that previous implementations were only approximations of the metric and not scalable above 8,000 points[2], which is potentially problematic in the case of voxels of $64 \times 64 \times 64$.

At a certain point during processing, some of the cap-

[2] https://github.com/facebookresearch/pytorch3d/issues/211

(a) "A hammock that is typical length and very slim. it does not have a headboard."

(b) "A clock that is short. it has a screen a frame and a base."

Figure 4: Misaligned captions and shapes.

tions became misaligned with the ground truth. Thankfully, the misalignment seem to have happened after training is done, so the resulting models from BERT and GRU does not seem affected. That said, sometimes the ground truth appears to be very far off from both BERT and GRU, although BERT and GRU consistently generate similar looking models. Refer to Figure 4 for examples when this does happen.

The quality of the captions didn't seem to be particularly high nor detailed. Perhaps with more descriptive, longer captions, the performance of the proposed pipeline would be better.

References

[1] Angel X. Chang, Thomas A. Funkhouser, Leonidas J. Guibas, Pat Hanrahan, Qi-Xing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, Jianxiong Xiao, Li Yi, and Fisher Yu. Shapenet: An information-rich 3d model repository. *CoRR*, abs/1512.03012, 2015. 1, 2

[2] Kevin Chen, Christopher B. Choy, Manolis Savva, Angel X. Chang, Thomas A. Funkhouser, and Silvio Savarese. Text2shape: Generating shapes from natural language by learning joint embeddings. *CoRR*, abs/1803.08495, 2018. 1

[3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018. 1, 3

[4] Tyler Habowski. Ai for 3d generative design, Mar 2020. 1, 2, 3, 4

[5] Ajay Jain, Ben Mildenhall, Jonathan T. Barron, P. Abbeel, and Ben Poole. Zero-shot text-guided object generation with dream fields. *ArXiv*, abs/2112.01455, 2021. 1

[6] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. 3

[7] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. 3

[8] Kaichun Mo, Paul Guerrero, Li Yi, Hao Su, Peter Wonka, Niloy J. Mitra, and Leonidas J. Guibas. Structurenet: Hierarchical graph networks for 3d shape generation. *CoRR*, abs/1908.00575, 2019. 1

[9] Kaichun Mo, Shilin Zhu, Angel X. Chang, Li Yi, Subarna Tripathi, Leonidas J. Guibas, and Hao Su. Partnet: A large-scale benchmark for fine-grained and hierarchical part-level 3d object understanding. *CoRR*, abs/1812.02713, 2018. 1, 2

[10] Alex Nichol, Prafulla Dhariwal, Aditya Ramesh, Pranav Shyam, Pamela Mishkin, Bob McGrew, Ilya Sutskever, and Mark Chen. GLIDE: towards photorealistic image generation and editing with text-guided diffusion models. *CoRR*, abs/2112.10741, 2021. 2

[11] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, 2021. 1

[12] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with clip latents, 2022. 1, 2

[13] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. Zero-shot text-to-image generation, 2021. 1, 2

[14] Nikhila Ravi, Jeremy Reizenstein, David Novotny, Taylor Gordon, Wan-Yen Lo, Justin Johnson, and Georgia Gkioxari. Accelerating 3d deep learning with pytorch3d. *arXiv:2007.08501*, 2020. 4

[15] Aditya Sanghi, Hang Chu, J. Lambourne, Ye Wang, Chin-Yi Cheng, and Marco Fumero. Clip-forge: Towards zero-shot text-to-shape generation. *ArXiv*, abs/2110.02624, 2021. 1

[16] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In

Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pages 38–45, Online, Oct. 2020. Association for Computational Linguistics. 3